



Figure 1: SIMD and SISD operations (source: MDN Mozilla)

Table 2

Feature	Brief description
SIMD	New methods that allow vector arithmetic.
Async functions	This is related to the <i>Promises</i> feature, which got introduced in ES6. <i>Promises</i> and the <i>async</i> functions will be covered in future issues.
<i>Object.values/Object.entries</i>	Like <i>Object.keys()</i> , this returns the properties of an object as an array to iterate or serialise
String padding	New string methods are <i>padStart()</i> and <i>padEnd()</i> . These will fill strings with a pattern.
Trailing commas in function parameter lists and calls	The last property or element does not have a comma before the closing brace. Allowing it ensures the convenience of rearranging the last element, without the overhead of comma adjustment.
<i>Object.getPrototypeOfDescriptors</i>	This method helps in copying between two objects and overcomes some limitations of <i>Object.assign()</i> .

the example given in Figure 1, this might look very trivial. But its applications and benefits are many.

Vector arithmetic improves performance and also saves power on devices. The applications that benefit are 3D graphics, cryptography, signal processing, etc. From a Web development point of view, it improves WebGL, HTML5 Canvas rendering, animation and *ASM.js*.

Here are the SIMD APIs:

SIMD.Float32x4() is the constructor to define array of values. Note that it takes only 4 values.

```
var x = SIMD.Float32x4(1.0, 1.5, 2.0, 2.5);
var y = SIMD.Float32x4(5.0, 5.5, 10.0, 10.5);
```

You can perform the addition operation using a single method that takes two vectors.

```
var z = SIMD.Float32x4.add(x,y);
```

Value of *z* is [6.0, 7.0, 12.0, 13.0]

Javascript/Node in demand

Though the original purpose of JavaScript was for interactivity in the Web browser, it has now extended its footprint to the server side through Node.js. Thanks to the contributions of Douglas Crockford, who made JSON the primary data exchange format. There is widespread adoption of Node.js in enterprises, not just in the presentation layer, but also in the business logic and persistence layers. Companies like PayPal, Netflix, GoDaddy, Mastercard and others have moved their production workloads to the Node environment. They have migrated in a phased manner from Java to JavaScript. There are a few startups that are laser focused on bringing Node to the enterprise.

All these are vital indications of where we are heading and the prominence that JavaScript will continue to command. Just learning one programming language like JavaScript is helping developers to be developers of full stacks and gain mastery in them.

According to Forrester Research, "By 2017, JavaScript will be the most in-demand

language skill in application development."

Next month, we will continue to explore the features of ES6, including Promises, Iterators and Generators. 

References

- [1] ECMAScript - current proposals and future champions: <https://github.com/tc39/ecma262>
- [2] TC39 Process, which explains the stages in detail: <https://tc39.github.io/process-document/>
- [3] ES2016 features - Array.prototype.includes(): <http://www.2ality.com/2016/02/array-prototype-includes.html>
- [4] ES2016 features - exponentiation operator: <http://www.2ality.com/2016/02/exponentiation-operator.html>
- [5] SIMD article on Intel open source website: <https://01.org/node/1495>

By: Janardan Revuru

The author believes in the vision of 'JavaScript Everywhere' and encourages developers to learn pure JavaScript in addition to frameworks. He is a co-organiser of the 'JavaScript meetup - Bangalore' group at <http://www.meetup.com/JavaScript-Meetup-Bangalore/>